

HARFANG HG Lua macOS Signing and Notarization Plan

Date: 2026-05-31

Executive Summary

Signing and notarizing the HG Lua macOS runtime is feasible, but the current runtime layout is not ready for a proper Apple distribution pass yet.

The current blockers are concrete:

- the shipped `lua`, `launcher`, and dylibs are ad hoc signed, not Developer ID signed
- Gatekeeper currently rejects the runtime
- the current macOS launch path is a shell script (`3-start.sh`), not a native `.app` bundle entry point
- the current packaging script zips a loose folder and copies the whole `bin/` tree, including developer tools
- the FFmpeg-related dylibs still reference Homebrew paths under `/opt/homebrew/...`, so the runtime is not self-contained

Recommended outcome:

- create a real macOS `.app` bundle for the user-facing HG Lua application
- use the native `launcher` executable as the app entry point
- make the runtime self-contained with only bundled non-system libraries
- sign all nested code with a `Developer ID Application` certificate
- notarize the exported archive with `notarytool`
- staple the ticket to the `.app` and optionally to a `.dmg`

Minimal fallback:

- sign every Mach-O file in the existing loose folder
- notarize a ZIP of that folder

That fallback is faster but not ideal. It keeps the shell-script launch model, cannot be stapled as cleanly, and is not the best Finder/Gatekeeper experience for end users.

Scope

This note is about the HG Lua macOS runtime currently visible in:

- `harfang3d/languages/hg_lua/CMakeLists.txt`
- `interactive-book/app/bin/hg_lua-macos-arm64/`
- `interactive-book/app/3-start.sh`
- `interactive-book/app/build-package.py`

It focuses on outside-the-Mac-App-Store distribution through Developer ID and Apple notarization.

Current Repository State

1. The runtime is built as loose executables and dylibs

`harfang3d/languages/hg_lua/CMakeLists.txt` installs:

- `lua`
- `launcher`
- `harfang.so`
- additional runtime dependencies under `hg_lua/`

The current sample macOS entry point is:

- `interactive-book/app/3-start.sh`

That script sets `LUA_CPATH` and directly executes:

- `./bin/hg_lua-macos-arm64/lua main.lua`

This is workable for local development, but it is not the right user-facing distribution shape for a notarized macOS product.

2. The current package is a ZIP of a loose folder

`interactive-book/app/build-package.py` copies the entire `bin/` tree and then creates a ZIP. That means the shipped payload currently includes much more than the runtime actually needed by end users, including toolchain executables under `harfang/assetc/`.

That increases signing surface and increases the chance of distributing tools that still depend on local Homebrew libraries.

3. The current code signatures are ad hoc

Local inspection of:

- `interactive-book/app/bin/hg_lua-macos-arm64/lua`
- `interactive-book/app/bin/hg_lua-macos-arm64/launcher`
- `interactive-book/app/bin/hg_lua-macos-arm64/libavcodec.61.19.101.dylib`

shows `Signature=adhoc` and no Team ID.

This is enough for local binaries to exist on disk, but it is not enough for Gatekeeper trust or notarization.

4. Gatekeeper currently rejects the runtime

Local `spctl -a -vv --type execute` on the shipped `lua` binary returns `rejected`.

So this is not a “final verification” problem. The full Developer ID and notarization workflow still needs to be added.

5. The FFmpeg side is not self-contained yet

The current runtime bundle includes FFmpeg-related files such as:

- `hg_ffmpeg.dylib`

- `libavcodec*.dylib`
- `libavformat*.dylib`
- `libavutil*.dylib`
- `libswscale*.dylib`
- `libswresample*.dylib`

However, local `otool -L` inspection shows that these files still reference absolute Homebrew install names under `/opt/homebrew/...`, including transitive dependencies outside the shipped folder.

This is a distribution blocker independent of notarization:

- notarization does not make broken loader paths acceptable
- a user machine cannot be expected to have the same Homebrew layout
- a proper app bundle or runtime folder must be closed over all non-system dependencies

What Apple Requires

For outside-the-App-Store macOS distribution, Apple's current requirements are straightforward:

- sign the software with a valid `Developer ID` certificate
- use `Developer ID Application` for apps and app-style executables
- enable Hardened Runtime for app and command-line targets that are notarized
- include a secure timestamp in the signature
- submit the software to the Apple notary service
- use `notarytool`, not the deprecated `altool`

Apple also distinguishes between certificate types:

- `Developer ID Application` signs apps
- `Developer ID Installer` signs installer packages

For HG Lua, the normal path is `Developer ID Application`. A `Developer ID Installer` certificate is only needed if you decide to ship a signed `.pkg`.

Recommended Distribution Target

Recommended: real `.app` bundle

For a user-facing Lua application, the recommended target is:

- `HG Lua.app`

with a standard structure such as:

```
HG Lua.app/  
  Contents/  
    Info.plist  
    MacOS/  
      HG Lua  
    Frameworks/  
      harfang.so
```

```
liblua54.dylib
libglfw3.dylib
hg_ffmpeg.dylib
...
Resources/
  assets_compiled/
  projects/
  main.lua
  launcher.json or bootstrap.json
  AppIcon.icns
```

Why this is the right target:

- Gatekeeper understands app bundles well
- **stapler** can attach a notarization ticket to a signed executable bundle
- Finder UX is normal
- it removes the shell-script launcher as the user entry point
- it creates a clean place for bundle metadata, icon, versioning, and resources

Strong recommendation: use **launcher** as the app executable

The repo already builds a native **launcher** target. That should be the user entry point for a signed/notarized macOS app.

Reasons:

- native executable entry point
- signable and notarizable
- avoids shipping a shell script as the primary launch surface
- aligns better with standard macOS application packaging

If you continue to expose raw **lua** as the main entry point, you can still sign and notarize it, but it is the weaker product shape.

Alternative for SDK/tool distribution: signed **.pkg**

If the target is not an end-user app but a developer-facing runtime or CLI package, a signed **.pkg** is also a legitimate distribution form.

That path requires:

- **Developer ID Installer**
- **productbuild --sign**
- notarization of the **.pkg**
- stapling of the **.pkg**

This is a better fit for an SDK or toolchain than for an interactive content player.

Minimal Fallback

If you want the fastest path with the fewest repo changes:

1. Keep the existing `hg_lua-macos-arm64/` folder layout.
2. Make the runtime self-contained.
3. Sign every Mach-O file in that folder tree.
4. ZIP the folder.
5. Submit the ZIP to notarization.

This can work, but it has real downsides:

- no proper `.app` bundle identity
- primary launch remains a shell script
- `stapler` does not staple a plain ZIP or a loose folder
- offline trust experience is worse than a stapled bundle or DMG

This should be treated as a stopgap, not the long-term result.

Required Technical Work

Step 1. Trim the distributable payload

Before signing anything, define what is actually shipped to end users.

Recommended rule:

- do not ship the whole `bin/` tree by default

For an end-user app, include only:

- the app executable (`launcher` recommended)
- the Lua runtime if still required
- `harfang.so`
- runtime dylibs actually loaded at execution time
- plug-ins actually used by the product such as `hg_ffmpeg.dylib` and `audio_xmp.dylib`
- app resources and assets

Do not ship these unless the final product truly needs them:

- `assetc`
- converter tools
- authoring toolchains
- build-time helper executables

This reduces:

- signing scope
- notarization scope
- bundle size
- risk from stray third-party dependencies

Step 2. Make the runtime self-contained

This is mandatory before signing.

The current FFmpeg-related dylibs still point at absolute Homebrew paths. That must be fixed by one of these approaches:

- rebuild FFmpeg and dependent plug-ins with relocatable install names
- rewrite install names with `install_name_tool`
- vendor every transitive non-system dylib into the app and point references at `@rpath` or `@loader_path`

Success condition:

- `otool -L` on every shipped executable, `.so`, and `.dylib` only shows bundled paths using `@rpath` or `@loader_path`
- `otool -L` on every shipped executable, `.so`, and `.dylib` only shows Apple system libraries under `/System/Library` or `/usr/lib` for non-bundled dependencies

Anything still referencing:

- `/opt/homebrew/...`
- `/usr/local/...`
- build-machine absolute paths

means the runtime is not yet releasable.

Step 3. Add app metadata

For the recommended `.app` path, add:

- `Info.plist`
- a stable bundle identifier such as `com.harfang.hg-lua.<product>`
- `CFBundleExecutable`
- version fields
- app icon if this is user-facing

If you use `launcher` as the entry point, also define the resource-side launch configuration cleanly, for example through `launcher.json` or `bootstrap.json`.

Step 4. Acquire Apple signing assets

You need:

- an active Apple Developer Program membership
- an Apple team with permission to create Developer ID certificates
- a `Developer ID Application` certificate
- optionally a `Developer ID Installer` certificate if shipping `.pkg`

Recommended automation credential for notarization:

- App Store Connect API key used through `notarytool`

Fallback credential:

- Apple ID + app-specific password stored through `notarytool store-credentials`

Step 5. Prepare Hardened Runtime

Sign the main executable with Hardened Runtime enabled.

Important points for HG Lua:

- start with no extra entitlements
- do not add JIT entitlements unless you actually use JIT
- do not disable library validation unless runtime behavior proves it is needed
- keep exceptions as narrow as possible

For this runtime, the default expectation should be:

- all bundled libraries are signed by the same Team ID
- no extra hardened runtime exceptions should be necessary

If something breaks after enabling Hardened Runtime, debug the actual cause before adding exceptions.

Step 6. Sign bottom-up

Do not rely on `codesign --deep` as the signing strategy.

Recommended order:

1. Sign nested dylibs, `.so` files, and plug-ins.
2. Sign helper executables inside the bundle.
3. Sign the main executable with Hardened Runtime.
4. Sign the outer `.app` bundle.

Typical command shape:

```
IDENTITY="Developer ID Application: Example Company (TEAMID)"

codesign --force --sign "$IDENTITY" --timestamp path/to/libfoo.dylib
codesign --force --sign "$IDENTITY" --timestamp path/to/harfang.so
codesign --force --sign "$IDENTITY" --timestamp path/to/helper_tool

codesign --force --sign "$IDENTITY" --timestamp --options runtime \
  path/to/HG\ Lua.app/Contents/MacOS/HG\ Lua

codesign --force --sign "$IDENTITY" --timestamp --options runtime \
  path/to/HG\ Lua.app
```

Notes:

- the main executable gets Hardened Runtime
- nested libraries normally do not need entitlements files
- shared libraries inherit the entitlements of their host executable

Step 7. Verify locally before notarization

Run this before upload:

```
codesign --verify --deep --strict --verbose=2 "HG Lua.app"
```

Also run a real launch test on a machine or user environment that does not depend on your local Homebrew setup.

For dependency verification, also run:

```
otool -L "HG Lua.app/Contents/MacOS/HG Lua"  
otool -L "HG Lua.app/Contents/Frameworks/hg_ffmpeg.dylib"
```

Step 8. Notarize with `notarytool`

Store credentials once:

```
xcrun notarytool store-credentials hg-notary \  
  --key /path/to/AuthKey_ABC1234567.p8 \  
  --key-id ABC1234567 \  
  --issuer 11111111-2222-3333-4444-555555555555
```

Then submit the archive:

```
ditto -c -k --keepParent "HG Lua.app" "HG Lua.app.zip"  
  
xcrun notarytool submit "HG Lua.app.zip" \  
  --keychain-profile hg-notary \  
  --wait
```

If notarization fails:

- fetch the notary log
- fix the underlying signing or packaging issue
- resubmit

Do not normalize failure by retrying blindly.

Step 9. Staple the ticket

After acceptance:

```
xcrun stapler staple "HG Lua.app"  
xcrun stapler validate "HG Lua.app"
```

```
spctl -a -vv --type execute "HG Lua.app"
```

Important limitation:

- **stapler** supports executable bundles, signed flat installer packages, and UDIF disk images
- it does not staple a plain loose folder
- it does not staple a ZIP archive

That is one more reason the **.app** path is the right target.

If your final download is a ZIP:

1. notarize the zipped **.app**
2. staple the **.app**
3. rebuild the final ZIP from the stapled **.app**

If your final download is a DMG:

1. create the **.app**
2. sign and notarize the DMG or notarize the zipped **.app**
3. staple the **.app** and optionally the DMG

What Can Be Automated

The following should be fully scriptable in the repo or in CI:

- assembling the release-only runtime payload
- creating the **.app** bundle directory structure
- generating **Info.plist** from templates
- copying resources into **Contents/Resources**
- copying bundled libraries into **Contents/Frameworks**
- rewriting install names to **@rpath** or **@loader_path**
- scanning the bundle for Mach-O files
- signing in deterministic bottom-up order
- running **codesign**, **spctl**, and **otool** validation steps
- zipping the **.app**
- submitting to notarization with **notarytool**
- waiting for notarization completion
- downloading and surfacing the notary log on failure
- stapling and validating the result
- failing CI when any signature, loader-path, or notarization step fails

Recommended implementation shape:

- add a dedicated macOS packaging/signing script
- make it run after the build output is frozen
- keep signing credentials outside the repo
- use CI secrets for the notary API key or a pre-provisioned keychain profile

What You Need To Do Manually

These parts cannot be meaningfully automated away:

- enroll in the Apple Developer Program
- pay the membership fee and keep it active
- accept Apple legal agreements
- have the Account Holder create or approve Developer ID certificates
- export/import the signing identity into the machine or CI keychain
- create the App Store Connect API key or app-specific password used by **notarytool**
- decide the final bundle identifier and product naming
- make the first pass on Hardened Runtime exceptions if runtime behavior breaks
- renew or revoke certificates when organizational policy requires it

For an organization account, the **Account Holder** role is relevant for Developer ID certificate creation according to Apple's current documentation.

What Is Partly Automatable

These are mostly one-time manual tasks followed by automation:

- creating a dedicated CI keychain and importing the certificate
- running **notarytool store-credentials** on the build machine
- generating a release icon and final **Info.plist** values
- splitting developer-only tools from runtime payload

Cost

Apple-side direct cost

As of 2026-05-31, Apple documents:

- Apple Developer Program: **99 USD/year**
- Apple Developer Enterprise Program: **299 USD/year**

For normal outside-the-Mac-App-Store distribution of HG Lua, the standard **99 USD/year** Apple Developer Program is the relevant one.

There is no separate Apple per-submission notarization fee listed in the cited Developer Program and notarization documentation. The practical assumption is that notarization is part of the Developer ID distribution workflow covered by membership.

Possible extra direct cost:

- local tax/VAT depending on region

Usually not needed:

- Enterprise Program membership

Engineering effort

If the goal is only "make the current loose folder pass signing/notarization," the work is small in theory but misleading in practice because the runtime is not yet self-contained.

Rough estimate for this repo:

- release payload cleanup: 0.5 to 1 day
- FFmpeg / loader-path cleanup to remove Homebrew dependencies: 1 to 3 days
- .app bundle assembly and metadata: 0.5 to 1.5 days
- signing + notarization scripting: 0.5 to 1 day
- CI integration and secrets wiring: 0.5 to 1 day

Total realistic initial effort:

- 2 to 5 engineer-days for a solid local workflow
- 3 to 6 engineer-days if CI and final release packaging are included

After that, each release should be mostly automated.

Infrastructure cost

You need a macOS machine or macOS CI runner for:

- codesign
- stapler
- final local verification

If you already build on macOS, there is no additional Apple-side cost here. CI runner cost depends on the provider and is outside the scope of this note.

Recommended Repo Changes

High priority

1. Add a release-only macOS packaging script for HG Lua.
2. Stop copying the entire bin/ tree into user-facing packages.
3. Build a real .app bundle around launcher.
4. Fix all FFmpeg and related dylib install names.
5. Add a signing script that signs bottom-up.
6. Add a notarization script that uses notarytool.

Medium priority

1. Add a CI job for signed/notarized release artifacts.
2. Add a clean entitlements file only if runtime exceptions are proven needed.
3. Optionally add DMG or PKG production after the .app path is stable.

Acceptance Checklist

The work is done when all of the following are true:

- otool -L shows no Homebrew or other build-machine absolute paths
- all shipped non-system code is signed with the same Team ID
- the main executable is signed with Hardened Runtime
- codesign --verify --deep --strict passes

- `spctl -a -vv --type execute` accepts the app
- notarization returns `Accepted`
- `stapler validate` passes on the final bundle or installer
- the product launches on a clean macOS machine without local Homebrew
- the release process is repeatable from one script or one CI workflow

Bottom Line

The main work is not the `notarytool` call. The main work is:

- packaging HG Lua as a real macOS product
- making the runtime self-contained
- signing all nested code correctly

Once that is done, notarization is straightforward and should be automated.

For this repo, the recommended path is:

- move from shell-script + loose folder distribution to `launcher + .app`
- trim the shipped runtime
- fix FFmpeg loader paths
- automate signing, notarization, and stapling

Sources

Apple official references consulted on 2026-05-31:

- Apple Developer Program enrollment and pricing:
<https://developer.apple.com/help/account/membership/program-enrollment>
- Apple Developer Program overview: <https://developer.apple.com/programs/>
- Developer ID certificates:
<https://developer.apple.com/help/account/certificates/create-developer-id-certificates/>
- Notarizing macOS software before distribution:
<https://developer.apple.com/documentation/security/notarizing-macos-software-before-distribution>
- Customizing the notarization workflow:
<https://developer.apple.com/documentation/security/customizing-the-notarization-workflow>
- Hardened Runtime: <https://developer.apple.com/documentation/security/hardened-runtime>

Local repo observations were also used from:

- `harfang3d/languages/hg_lua/CMakeLists.txt`
- `interactive-book/app/3-start.sh`
- `interactive-book/app/build-package.py`
- `interactive-book/app/bin/hg_lua-macos-arm64/`